Réponses et Guide pour le TP3 : LiDAR

Ce document contient toutes les réponses aux questions théoriques du sujet ainsi que les portions de code à compléter dans les différents fichiers pour vous faire gagner du temps.

Partie 2: C'est quoi un LiDAR 2D ?

• **Dans quel plan en 2D sont effectuées les mesures ?** Les mesures sont effectuées dans le plan horizontal (parallèle à la rotation de la tête du LiDAR).

• **Sachant que la tête du LiDAR tourne constamment, comment l'envoi et la réception du signal lumineux se fait-il ?** Le LiDAR possède un système de transmission sans fil mixte (optique pour les données et couplage magnétique/sans contact pour l'alimentation). Cela lui permet de tourner à 360° en continu sans enrouler de câbles (contrairement à l'utilisation d'un *slip ring* classique qui s'use avec le temps).

• **Comment la distance est calculée à partir du signal lumineux ?** Ce modèle (RPLIDAR A1) utilise le principe de télémétrie par **triangulation laser**. Un laser infrarouge modulé est émis, rebondit sur l'obstacle, et le reflet est lu par une caméra/capteur d'acquisition haute vitesse. La distance est déduite grâce à l'angle sous lequel le rayon réfléchi est observé.

Partie 3: Mise en place

Manipulation matérielle : 1. Branchez votre LiDAR en USB. 2. Identifiez le port avec la commande `ls /dev/*USB*`. (Généralement `/dev/ttyUSB0`). 3. **Modification dans `config/constants.py` :**

```
PORT_LIDAR = "/dev/ttyUSB0" # (ou autre selon votre machine)
```

Partie 4: You spin me right round, baby, right round

1. Portées du LiDAR (`config/constants.py`) : Selon la documentation (Datasheet, p.8), la portée minimale (typical) est de 0.15m et maximale de 12m.

```
LIDAR_MIN_RANGE = 150       # 150 mm
LIDAR_MAX_RANGE = 12000     # 12000 mm
```

2. Exécution de `single_scan.py` : • **Combien de mesures contient le scan ?** Environ 1450 mesures (selon la documentation pour une fréquence de 5.5 Hz). Exécutez le script pour voir le chiffre exact pour votre scan. • **Quelles**

informations pour chaque mesure, format, unité ? Chaque mesure est un "tuple" de 3 éléments: `(Qualité, Angle en degrés, Distance en mm)`.

3. **Modifications dans `sensors/LIDAR/lidar.py` :**

- **`polar_to_cartesian(self, scan)` : Note sur les axes :** Si x=0 pour 0° et 180°, et y=0 pour 90° et 270°, cela signifie que l'angle mesure la position par rapport à l'axe Y. L'axe à 0° se trouve généralement en face du capteur. Le LiDAR tourne dans le sens horaire.

```python
def polar_to_cartesian(self, scan):
    points = []
    for quality, angle, distance in scan:
        rad = math.radians(angle)
        # x=0 en 0 et 180 -> la composante est sin(angle)
        x = distance * math.sin(rad)
        # y=0 en 90 et 270 -> la composante est cos(angle)
        y = distance * math.cos(rad)
        points.append((x, y))
    return points
```

- **`filter_quality(self, scan)` :**

```python
def filter_quality(self, scan):
    # On garde les mesures où la qualité est >= self.quality_threshold
    return [meas for meas in scan if meas[0] >= self.quality_threshold]
```

- **`filter_angle(self, scan)` :**

```python
def filter_angle(self, scan):
    filtered = []
    for meas in scan:
        quality, angle, distance = meas
        # Gérer le cas où min > max (ex: 270° à 90°)
        if self.min_angle <= self.max_angle:
            if self.min_angle <= angle <= self.max_angle:
                filtered.append(meas)
        else:
            if angle >= self.min_angle or angle <= self.max_angle:
                filtered.append(meas)
    return filtered
```

- **`filter_range(self, scan)` :**

```python
def filter_range(self, scan):
    # On garde les mesures avec une distance comprise entre min_range et max_range
    return [meas for meas in scan if self.min_range <= meas[2] <= self.max_range]
```

Partie 5: Au millimètre près ?

Informations Datasheet : • **Fréquence** : 5.5 Hz par défaut (configurable entre 1 et 10 Hz). • **Précision** : $< 1\%$ de la distance • **Résolution de distance** : < 0.5 mm • **Résolution angulaire** : $<= 1°$ • **Différence Précision / Résolution** : La *résolution* est le plus petit pas mesurable (ex: le capteur peut voir une différence de 0.5mm). La *précision* est l'écart entre la valeur mesurée et la réalité (l'erreur max).

Code à compléter dans `scripts/measure_distance.py` :

```
    for timestamp, scan, points in scan_iterator:
        # Extraire les distances. Le scan est déjà filtré par le scan_iterator.
        distances = [meas[2] for meas in scan]

        if len(distances) > 0:
            mean_distance = np.mean(distances)
            std_distance = np.std(distances)
        else:
            mean_distance = 0
            std_distance = 0

        print(f"[{timestamp:.2f}] Nb points: {len(distances)}, "
              f"Dist Moyenne: {mean_distance:.2f} mm, "
              f"Ecart-Type: {std_distance:.2f} mm")

        yield timestamp, mean_distance, std_distance
```

• **Tests à réaliser en TP :** Testez avec divers paramètres (`MIN_DETECTION_ANGLE` et `MAX_DETECTION_ANGLE` dans config) face à un mur puis une paroi vitrée. **Reponse attendue :** La fenêtre causera de nombreuses erreurs car le laser passera au travers ou sera mal réfléchi. Plus l'angle d'inclinaison par rapport au laser est grand, plus la précision diminue (la tâche lumineuse s'élargit). L'écart-type augmentera avec la distance.

Partie 6: WEEEU WEEEU, CECI EST UNE DESCENTE DE POLICE !

1. Détecteur de proximité (`scripts/proximity_sensor.py`) : Le paramètre `alert_distance` est passé à la fonction `proximity_monitor`. L'objectif est de sonner dès qu'un objet est plus proche.

```
def proximity_monitor(scan_iterator, buzzer, alert_distance):
    for timestamp, scan, points in scan_iterator:
        distances = [meas[2] for meas in scan]
        # Active le buzzer si au moins une mesure est plus petite que l'alerte.
```

```
        if any(d < alert_distance for d in distances):
            buzzer.on()
        else:
            buzzer.off()
```

2. Détecteur de mouvement (`scripts/motion_sensor.py`) : Il faut utiliser `scan_to_bins` pour grouper les zones et comparer les changements d'une étape à la suivante. Si la différence avec l'étape précédente dépasse `motion_threshold`, on déclenche le buzzer.

```
def motion_monitor(scan_iterator, buzzer, max_range=LIDAR_MAX_RANGE, motion_threshold=200.0,
    previous_signature = None
    for timestamp, scan, points in scan_iterator:

        signature = scan_to_bins(
            scan,
            num_bins=num_bins,
            default_distance=max_range
        )

        if previous_signature is not None:
            # Calcule la différence absolue entre chaque bin
            diffs = []
            for s, p in zip(signature, previous_signature):
                if s is not None and p is not None:
                    diffs.append(abs(s - p))

            # Si un mouvement significatif est détecté
            if diffs and any(d > motion_threshold for d in diffs):
                buzzer.on()
            else:
                buzzer.off()

        previous_signature = signature
```

Bon TP ! Vous n'avez plus qu'à manipuler le matériel (et le terminal pour lancer les scripts) !